

Centro Federal de Educação Tecnológica (CEFET-MG)
Departamento de Computação (DECOM)
Campus Nova Gameleira

Prof. Eduardo Cunha Campos

Lista 02 de Exercícios

Aluno(a): Gustavo de Assis Xavier

Matrícula: 20223003379

Questão 01. Em relação à refatoração, assinale com V (verdadeiro) ou F (falso) as afirmações abaixo.

- () O melhor momento para se refatorar um código é durante os testes de aceitação, pois o cliente tem interesse em um código de qualidade.
- () Um dos passos da refatoração é a aplicação dos testes que verificarão sua implementação.
- () Rotinas muito longas e código duplicado são exemplos de *bad smells*. () Refatorações são modificações no código que são simples a ponto de não gerarem nenhum efeito prático.
- () Um código que já foi refatorado uma vez não precisará ser refatorado no futuro, pois já atende aos critérios de qualidade exigidos.
- () A refatoração de um código implica apenas a melhoria de sua qualidade interna e não deve afetar sua funcionalidade original.

A sequência correta de preenchimento dos parênteses, de cima para baixo, é:

- A) F–V–F–V–F–V.
- B) V–F–V–F–V–F.
- C) F–V–V–F–F–V.**
- D) F–V–F–V–V–F.
- E) V–F–V–F–V–V.

Questão 02. Acerca do tema refatoração de *software*, considere as afirmativas.

- I. A refatoração busca evoluir o projeto e código-fonte de um sistema de *software* para se alcançar alta coesão, isto é, suas classes devem possuir conjuntos extensos de responsabilidades.
- II. A refatoração busca evoluir o projeto e código-fonte de um sistema de *software* para alcançar baixo acoplamento, isto é, a colaboração entre as classes deve ser mantida em um nível mínimo aceitável.
- III. A refatoração é o processo de mudar um sistema de *software* de tal forma que não altere o comportamento externo do código-fonte, embora melhore sua estrutura interna.

Está(ão) correta(s):

- A) I, II e III.

- B) I e II, somente.
- C) I e III, somente.
- D) III, somente.
- E) II e III, somente.**

Questão 03. Quanto à refatoração (*refactoring*), analise as afirmativas abaixo e assinale a alternativa correta.

I. Quando o prazo de entrega está próximo do final, deve-se fazer a refatoração. II. Quando o código está instável, e não funciona, é o melhor momento para refatorar. III. *EXtreme Programming* tem refatoração como uma de suas práticas. IV. Ela consiste em melhorar a estrutura do código de um sistema preservando as suas funcionalidades.

Estão corretas as afirmativas:

- A) Somente a I e II.
- B) Somente a II e III.
- C) Somente a III e IV.**
- D) Somente a I e III.
- E) I, II, III e IV.

Questão 04. Discorra sobre 4 técnicas de Reuso de *Software* amplamente adotadas no cenário do desenvolvimento moderno de *software*.

Bibliotecas reutilizáveis: Código que podem ser reutilizados em diferentes projetos.

Frameworks: Estruturas que fornecem uma base para o desenvolvimento de aplicações.

Componentes de terceiros: Pacotes ou módulos já prontos que podem ser integrados a sistemas, como APIs ou plugins.

Padrões de projeto (Design Patterns): Soluções para problemas comuns em projetos, como Singleton, Factory e Observer.

Questão 05. Na programação de computadores, um cheiro de código (do inglês, “*code smell*”) é qualquer característica no código-fonte de um programa que possivelmente indica um problema mais profundo. *Shotgun Surgery* e *Divergent Change* são exemplos de *bad smells*. O que consiste cada um desses *smells* e qual é a diferença entre eles?

Shotgun Surgery é quando uma responsabilidade está dividida em várias classes, ou seja, a lógica está espalhada. Já *Divergent Change* é quando há muitas responsabilidades em uma só classe/método.

Questão 06. O modelo ágil se tornou uma tendência em empresas que precisam agregar mais valor para seus clientes, responder mais rapidamente às mudanças de mercado, diminuir falhas e aumentar a cadência de inovações em seus projetos de *software*. O *Scrum* é uma metodologia de desenvolvimento ágil baseado no conceito de equipes auto-organizadas. Quais são os papéis presentes em equipes *Scrum*? Cite e explique as

responsabilidades de cada um dos papéis.

Product Owner (PO):

Gerencia o backlog do produto. Define prioridades, comunica a visão do produto e garante que a equipe esteja alinhada com os objetivos do cliente.

Scrum Master:

Garante que as práticas do Scrum sejam seguidas, remove impedimentos e promove um ambiente de trabalho eficiente.

Equipe de Desenvolvimento:

Grupo responsável por entregar os incrementos do produto a cada sprint. Desenvolver.

Questão 07. *Refactoring* é o processo que:

A) implementa todas as funcionalidades da camada de model para depois implementar as camadas de controller e de viewer, nos casos em que a arquitetura MVC é utilizada.

B) efetua mudanças em um código existente e funcional sem alterar seu comportamento externo, com o objetivo de aprimorar a estrutura interna do código.

C) inclui funcionalidades extras no código, com o intuito de aprimorá-lo. D) aprimora a extração e o refinamento iterativo dos requisitos do produto ainda na fase de planejamento do *software*, sendo considerado um valor na metodologia XP. E) estabelece os métodos, um após o outro, para depois definir as classes e suas abstrações e implementar as interfaces.

Questão 08. A Reengenharia de *Software*, conhecida também como renovação ou reconstrução, é o exame e alteração de um sistema de *software*, para reconstituí-lo em uma nova forma, e a subsequente implementação dessa nova forma. Um processo de reengenharia geralmente inclui alguma forma de engenharia reversa, seguida por uma forma de engenharia progressiva ou reestruturação. Descreva 3 tarefas da Reengenharia.

Engenharia reversa: Pegar um programa pronto e documentá-lo.

Refatoração: Melhorar o código sem alterar funcionalidade.

Adaptação: Preparar o software para novas tecnologias e integrações.

Questão 09. Programação extrema (XP), é considerada uma metodologia ágil pois se ajusta bem a pequenas e médias em desenvolvimento de software com requisitos vagos e em constante mudança. Integração contínua é uma prática que foi introduzida pela XP. O que consiste esta etapa? Cite alguns exemplos de servidores de integração contínua adotados atualmente nos projetos de *software*.

Integração contínua é a prática de integrar o trabalho de todos os desenvolvedores em um repositório compartilhado diversas vezes ao longo do tempo. Exemplos: GitHub

Actions, GitLab CI/CD, Jenkins.

Questão 10. Os *bad smells* presentes no código podem dificultar as atividades de manutenção do código-fonte. Essas estruturas precisam ser refatoradas para melhorar aspectos de legibilidade e compreensibilidade de código. Neste sentido, descreva a ocorrência de 4 tipos de *bad smells* que já foram catalogados na literatura. Para cada um deles, cite e explique qual é a ação de refatoração que deve ser feita para eliminá-lo.

Long Method:

Descrição: Métodos com muitas linhas de código, dificultando a leitura e a manutenção.

Solução: Extract Method: dividindo o método em partes menores e mais específicas.

Duplicated Code (Código Duplicado):

Descrição: Códigos semelhantes aparecem em locais diferentes.

Solução: Consolidar os trechos duplicados em um único método ou classe.

Large Class (Classe Grande):

Descrição: Classes que acumulam muitas responsabilidades, tornando-as difíceis de entender e modificar.

Solução: Extract Class: Separar responsabilidades em classes menores.

Feature Envy (Inveja de Recursos):

Descrição: Um método usa excessivamente os atributos e métodos de outra classe.

Solução: Move Method: mover o método para a classe onde ele faz mais sentido.